5.2.1.1 - מהי האסטרטגיה שמומשה עבור CleverPlayer ו - GeniusPlayer

CleverPlayer - מימשתי אסטרטגיה של ניסיון למלא שורות שלמות, פשוט מעבר ב2 לולאות, ניסיון למלא שורה, כלומר ננסה למלא את שורה 1 - עמודה ראשונה, שנייה וכו'...
אם השורה מלאה ועוד לא ניצחנו אז ממשיכים למלא את הבאה, וכן הלאה.
יש יותר סיכוי לנצח ככה מאשר בצורה רנדומלית ולכן השחקן החכם מצליח לנצח את הרנדומלי ביותר מ - 80% אחוז בלוח דיפולטיבי בטורניר של 10000 סיבובים (כשהנדרש היה 65%).
GeniusPlayer - מימשתי אסטרטגיה בה קודם כל השחקן בודק אם בתור הנוכחי הוא יכול להגיע ל3 סימונים רצוף, אז השחקן יסמן זאת, כי במשחק עם ערכים דיפולטיבים (כמו שנדרש באחוזים) זה אומר שהוא ינצח. אחרת יבדוק אם לשחקן שמולו יש אופציה במהלך הבא שלו להשלים במקום מסוים רצף של 3, אם כן יחסום אותו כדי להימנע מהפסד (שוב, רלוונטי ללוח הדיפולטיבי שנטען שישחקו בו).
לבסוף ירוץ בלולאה שמתחילה מהעמודה השנייה וינסה למלא כמו השחקן ה - Clever את העמודות (במקום השורות) לפי הסדר. ההתחלה שלו מעמודה 1 חוסמת את נצחונו של השחקן החכם ולכן מגיע ל100 אחוז ניצחון מולו (כאשר הנדרש היה 85%).
מול השחקן הרנדומלי הוא מגיע למעל 97% נצחון.

5.2.1.2 - היתרון בעיצוב התוכנה באופן שכל אחת ממחלקות השחקנים מממשת ממשק משותף

היתרון הוא שאפשר ליצור מתודת playTurn משותפת שמוגדרת בממשק הכללי, וכל מחלקת שחקן מממשת אותה בדרך הייחודית לה. כך מחלקת המשחק (Game) יכולה לנהל את התורות מבלי לדעת איזה סוג שחקן משחק באותו הרגע.
עיצוב זה מתבסס על עמוד התווך של הפולימורפיזם ב - OOP. פולימורפיזם מאפשר לנו להתייחס לאובייקטים ממחלקות שונות (כמו שחקן אנושי, רנדומלי, חכם או גאון) דרך ממשק אחיד (Player) כאשר בזמן הריצה המערכת יודעת להפעיל את המימוש הספציפי של אותו אובייקט.
כתוצאה מכך, הקוד מקיים גם את עקרון "פתוח-סגור": המערכת פתוחה להרחבה (ניתן להוסיף סוגי שחקנים חדשים בקלות על ידי מימוש הממשק), אך סגורה לשינויים (אין צורך לשנות אף שורת קוד במחלקות האחרות כדי לתמוך בשחקן החדש).
לכן אנו עומדים בעקרונות ה - OOP שלמדנו בכיתה.

5.2.1.3 - מחלקות נוספות

לא הוספתי מחלקות נוספות בתרגיל.